SIMATS SIMATS

Department of Computer Science and Engineering

Course Code & Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO1 – Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform
Course Outcome Assessed	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.
Duration	3 Days
Mode	Individual
Student Reg. No.	192411048
Student Name	CH.Purnima

Instructions: Answer all parts. Show the basic operation, every step of each derivation and the final asymptotic result, and interpret what each result means for the platform. Assume the platform's servers execute 10^8 basic operations per second.

IT APPLICATION-BASED PROBLEM

Scenario – Scalability Audit of "ShopSphere", an Online Shopping Platform: ShopSphere is a growing e-commerce platform whose product catalogue currently holds n = 100,000 products and is expected to reach n = 1,000,000 within a year. Before the expansion, the CTO has asked you, as the platform's algorithm engineer, to audit five back-end modules, predict how each will behave as n grows, and recommend which modules must be redesigned. The modules are described below.

Module M1 – Duplicate-Listing Detector: Sellers sometimes upload the same product twice. Every night, the following routine compares every product with every later product.

```
FOR i ← 0 TO n − 2 DO
   FOR j ← i + 1 TO n − 1 DO
      IF Product[i].title = Product[j].title AND Product[i].seller =
Product[j].seller THEN
         MarkDuplicate(j)
      END IF
   END FOR
END FOR
```

Module M2 – API Rate Limiter: The public API audits request windows of doubling size using the routine below. A team member claims that it is Θ(n log n) because the outer loop runs about log n times and the inner loop runs up to n times.

```
i ← 1
WHILE i < n DO
   FOR j ← 0 TO i − 1 DO
      Audit(Request[j])          // basic operation
   END FOR
   i ← i × 2
END WHILE
```

Module M3 – Product Search: Two versions exist. Version S1 scans the unsorted catalogue sequentially until the product is found. Version S2 searches a sorted product index by comparing

the key with the middle entry and discarding half the index after each comparison; its comparison count satisfies T(n) = T(n/2) + 1, T(1) = 1.

Module M4 – Recommendation Engine: The engine splits the customer–product data into four parts of size n/2 (because of replicated partitions) and merges the partial results in linear time: T(n) = 4T(n/2) + n. A proposed redesign removes the replication, giving T(n) = 2T(n/2) + n.

Module M5 – Sales Analytics Engine: The analytics engine splits the sales data into halves recursively and sorts the records by revenue at every level, giving T(n) = 2T(n/2) + n log n. Separately, profiling shows that the nightly report generator performs $f(n) = 3n^2 + 10n \log_2 n + 50$ operations.

(a) Understand the Problem: For each of the five modules, identify the input size and the basic operation that dominates the running time, explain where it occurs, and state how you expect its execution count to grow as n increases. Justify each choice briefly.

(b) Analyse the Iterative Modules: For M1, derive the exact number of comparisons as a function of n and state the time and space complexity. For M2, derive the exact number of Audit operations, decide whether the team member's Θ(n log n) claim is correct and give the tight bound. Verify your M2 result for n = 1,024.

(c) Solve a Recurrence by Substitution: Solve the recurrence for search version S2 using backward substitution, showing every intermediate step, and derive the exact closed form. State the time complexity and the space complexity of both an iterative and a recursive implementation.

(d) Apply the Master Theorem: For M4, apply the Master Theorem to both the current design and the proposed redesign, clearly identifying a, b, f(n), n^(log_b a) and the case used, and explain what happens to the running time when the data doubles. For M5, determine whether any of the three standard cases applies to T(n) = 2T(n/2) + n log n; if not, derive the complexity using the extended Master Theorem or a recursion tree.

(e) Analyse Best, Average and Worst Cases: For search versions S1 and S2, identify the input condition that produces the best, average and worst case and give the number of comparisons in each. For S1, derive the average number of comparisons when the probability that the searched product exists is p, and evaluate it for n = 1,000,000 and p = 0.5.

(f) Prove an Asymptotic Bound: Using the formal definitions, prove that the report generator's cost $f(n) = 3n^2 + 10n \log_2 n + 50$ is $\Theta(n^2)$ by finding constants c_1, c_2 and n_0, and show that f(n) is not O(n).

(g) Scalability Decision: Estimate the number of basic operations and the running time of M1, M2, S2, M4 (both designs) and M5 for n = 1,000,000. Identify the modules that will fail to meet a nightly processing window of 10 minutes, and recommend, with justification, which modules must be redesigned and how.

SCALABILITY AUDIT OF "SHOPSPHERE"

1. Introduction

ShopSphere is a growing online shopping platform whose product catalogue currently contains 100,000 products and is expected to reach 1,000,000 products within a year. The purpose of this scalability audit is to analyse the algorithms used in five important backend modules. The running time, space requirements, asymptotic complexity, recurrence relations, and scalability of each module are studied. The server is assumed to execute 10^8basic operations per second, and the maximum acceptable nightly processing time is 10 minutes.

(a) UNDERSTAND THE PROBLEM

For all the modules, $n$represents the number of products, requests, records, or data items being processed.

Module	Input Size	Basic Operation	Location	Expected Growth
M1 – Duplicate Listing Detector	n products	Comparing two products	Inner IF condition	Θ(n²)
M2 – API Rate Limiter	n requests	Audit(Request[j])	Inner loop	Θ(n)
M3 – Product Search S1	n products	Comparing search key	Sequential search	Θ(n)
M3 – Product Search S2	n products	Comparing with middle element	Binary search	Θ(log n)
M4 – Recommendation Engine	n data items	Recursive processing and merging	Recursive calls	Current: Θ(n²)
M4 – Proposed Design	n data items	Recursive processing and merging	Recursive calls	Θ(n log n)
M5 – Sales Analytics	n records	Sorting/processing at each level	Recursive levels	Θ(n log² n)

M1

The dominant operation is the comparison between two products. Every product is compared with all the products appearing after it. Therefore, the number of comparisons increases quadratically.

M2

The outer loop doubles the value of i, but the inner loop executes $1,2,4,8,$ ...times. The total of these values is linear rather than $n \log n$.

M3

S1 checks the products sequentially, so its running time can grow linearly. S2 removes half of the search space after every comparison, giving logarithmic running time.

M4

The current design creates four recursive subproblems of size $n/2$, resulting in quadratic complexity. The proposed design creates only two subproblems and gives $\Theta(n \log n)$.

M5

The recurrence contains $n \log n$work at every recursion level. Therefore, the total complexity becomes $\Theta(n\log^2 n)$.

(b) ANALYSE THE ITERATIVE MODULES

M1 – Duplicate Listing Detector

The loops compare every product with every later product.

For $i = 0$, the number of comparisons is:

$$n - 1$$

For $i = 1$:

$$n - 2$$

For $i = 2$:

$$n - 3$$

The total number of comparisons is therefore:

$$C(n) = (n-1) + (n-2) + \cdots + 2 + 1$$

Using the sum of the first $n - 1$integers:

$$C(n) = \frac{(n-1)n}{2}$$

Therefore,

$$\boxed{C(n) = \frac{n(n-1)}{2}}$$

Expanding:

$$C(n) = \frac{n^2 - n}{2}$$

The dominant term is n^2.

Therefore, the time complexity is:

$$\boxed{T(n) = \Theta(n^2)}$$

The algorithm uses only a constant amount of extra space apart from the input catalogue.

$$\boxed{S(n) = \Theta(1)}$$

M2 – API Rate Limiter

The values of $i$are:

$$1,2,4,8, \ldots$$

The total number of Audit operations is:

$$1 + 2 + 4 + 8 + \cdots + 2^k$$

where:

$$k = \lfloor \log_2(n-1) \rfloor$$

Using the geometric series formula:

$$1 + 2 + 4 + \cdots + 2^k = 2^{k+1} - 1$$

Therefore:

$$\boxed{A(n) = 2^{\lfloor \log_2(n-1) \rfloor + 1} - 1}$$

Hence:

$$\boxed{A(n) = \Theta(n)}$$

Checking the Team Member's Claim

The team member claims that the algorithm is $\Theta(n \log n)$. This is incorrect.

The inner loop does not execute $n$times during every iteration. Instead, the numbers of operations are:

$$1 + 2 + 4 + 8 + \cdots$$

Their total is linear.

Therefore:

$$\boxed{\text{M2 is } \Theta(n), \text{ not } \Theta(n \log n)}$$

Verification for $n = 1024$

The values of $i$are:

$$1,2,4,8,16,32,64,128,256,512$$

Therefore:

$$A(1024) = 1 + 2 + 4 + \cdots + 512$$
$$= 2^{10} - 1$$
$$\boxed{A(1024) = 1023}$$

Thus, the M2 module performs **1023 Audit operations** when $n = 1024$.

(c) SOLVE S2 USING SUBSTITUTION

The recurrence for S2 is:

$$T(n) = T(n/2) + 1$$

with:

$$T(1) = 1$$

Starting with:

$$T(n) = T(n/2) + 1$$

Substituting $T(n/2)$:

$$T(n) = T(n/4) + 1 + 1$$
$$T(n) = T(n/4) + 2$$

Substituting again:

$$T(n) = T(n/8) + 3$$

After $k$substitutions:

$$T(n) = T\left(\frac{n}{2^k}\right) + k$$

The recursion stops when:

$$\frac{n}{2^k} = 1$$

Therefore:

$$n = 2^k$$

Taking logarithm:

$$k = \log_2 n$$

Therefore:

$$T(n) = T(1) + \log_2 n$$

Since:

$$T(1) = 1$$

we get:

$$\boxed{T(n) = \log_2 n + 1}$$

Thus, the time complexity is:

$$\boxed{T(n) = \Theta(\log n)}$$

Iterative Implementation

An iterative binary search uses only a few variables.

$$\boxed{Space = \Theta(1)}$$

Recursive Implementation

A recursive implementation creates approximately $\log n$function calls.

$$\boxed{Space = \Theta(\log n)}$$

Therefore:

Implementation	Time Complexity	Space Complexity
Iterative S2	Θ(log n)	Θ(1)
Recursive S2	Θ(log n)	Θ(log n)

(d) APPLY THE MASTER THEOREM

M4 – Current Design

The recurrence is:

$$T(n) = 4T(n/2) + n$$

Comparing with:

$$T(n) = aT(n/b) + f(n)$$

we get:

$$a = 4,\ b = 2,\ f(n) = n$$

Now calculate:

$$n^{\log_b a} = n^{\log_2 4}$$
$$= n^2$$

Therefore:

$$n^{\log_b a} = n^2$$

and:

$$f(n) = n$$

Since $n$is polynomially smaller than n^2, Master Theorem Case 1 applies.

Therefore:

$$\boxed{T(n) = \Theta(n^2)}$$

Effect of Doubling the Data

If $n$becomes $2n$:

$$T(2n) = (2n)^2$$
$$= 4n^2$$

Therefore, doubling the data approximately makes the running time **four times larger**.

M4 – Proposed Redesign

The proposed recurrence is:

$$T(n) = 2T(n/2) + n$$

Therefore:

$$a = 2,\ b = 2,\ f(n) = n$$

Calculate:

$$n^{\log_b a} = n^{\log_2 2} = n$$

Thus:

$$f(n) = \Theta\left(n^{\log_b a}\right)$$

Master Theorem Case 2 applies.

Therefore:

$$\boxed{T(n) = \Theta(n \log n)}$$

When the data doubles, the running time increases approximately by a factor close to 2, with a small additional logarithmic increase.

The proposed design is therefore much more scalable than the current design.

M5 – Sales Analytics Engine

The recurrence is:

$$T(n) = 2T(n/2) + n \log n$$

Here:

$$a = 2,\ b = 2$$

and:

$$f(n) = n \log n$$

Calculate:

$$n^{\log_b a} = n^{\log_2 2} = n$$

The extra factor of $\log n$means that the basic three-case form of the Master Theorem does not directly apply.

Using the extended Master Theorem:

$$f(n) = \Theta\left(n^{\log_b a} \log^k n\right)$$

where:

$$k = 1$$

Therefore:

$$T(n) = \Theta\left(n^{\log_b a} \log^{k+1} n\right)$$
$$= \Theta(n \log^2 n)$$

Hence:

$$\boxed{T(n) = \Theta(n \log^2 n)}$$

(e) BEST, AVERAGE AND WORST CASES

S1 – Sequential Search

Best Case

The searched product is the first product.

$$\boxed{1 \text{ comparison}}$$

Average Case

If the product exists and can be at any of the $n$positions with equal probability:

$$C_{avg} = \frac{1 + 2 + \cdots + n}{n}$$
$$= \frac{\frac{n(n+1)}{2}}{n}$$
$$\boxed{C_{avg} = \frac{n+1}{2}}$$

Worst Case

The product is the last product or is not present.

$$\boxed{C_{worst} = n}$$

S2 – Binary Search

Best Case

The required product is the middle element.

$$\boxed{1 \text{ comparison}}$$

Average Case

The search requires logarithmically many comparisons.

$$\boxed{\Theta(\log n)}$$

Worst Case

The search continues until the final remaining portion.

$$\boxed{\Theta(\log n)}$$

For $n = 1{,}000{,}000$:

$$\lfloor \log_2(1,000,000) \rfloor + 1$$
$$= 19 + 1$$
$$\boxed{20 \text{ comparisons approximately}}$$

Average Comparisons for S1 When Existence Probability is p

If the product exists with probability p:

$$E[C \mid exists] = \frac{n+1}{2}$$

If it does not exist:

$$E[C \mid absent] = n$$

Therefore:

$$E[C] = p\left(\frac{n+1}{2}\right) + (1-p)n$$

Simplifying:

$$E[C] = n - \frac{p(n-1)}{2}$$

Therefore:

$$\boxed{E[C] = n - \frac{p(n-1)}{2}}$$

For:

$$n = 1{,}000{,}000,\ p = 0.5$$
$$E[C] = 1{,}000{,}000 - \frac{0.5(999,999)}{2}$$
$$= 750{,}000.25$$

Therefore:

$$\boxed{E[C] \approx 750{,}000 \text{ comparisons}}$$

This demonstrates the large advantage of S2 over S1 for a million-product catalogue.

(f) PROVE THE ASYMPTOTIC BOUND

The report generator has:

$$f(n) = 3n^2 + 10n\log_2 n + 50$$

We need to prove:

$$f(n) = \Theta(n^2)$$

For the lower bound:

$$f(n) = 3n^2 + 10n\log_2 n + 50$$

Since all terms are positive:

$$f(n) \geq 3n^2$$

Choose:

$$\boxed{c_1 = 3}$$

For the upper bound, when $n \geq 2$:

$$\log_2 n \leq n$$

Therefore:

$$10n\log_2 n \leq 10n^2$$

Also:

$$50 \leq 50n^2$$

Therefore:

$$f(n) \leq 3n^2 + 10n^2 + 50n^2$$
$$f(n) \leq 63n^2$$

Choose:

$$\boxed{c_2 = 63}$$

and:

$$\boxed{n_0 = 2}$$

Thus, for every $n \geq 2$:

$$3n^2 \leq f(n) \leq 63n^2$$

Hence:

$$\boxed{f(n) = \Theta(n^2)}$$

Proving that f(n) is Not O(n)

Assume that:

$$f(n) = O(n)$$

Then there must exist a constant $c$such that:

$$f(n) \leq cn$$

for sufficiently large n.

But:

$$f(n) \geq 3n^2$$

Therefore:

$$3n^2 \leq cn$$

Dividing by n:

$$3n \leq c$$

As $n$increases, $3n$becomes arbitrarily large. Therefore, no fixed constant $c$can satisfy this condition.

Hence:

$$\boxed{f(n) \notin O(n)}$$

(g) SCALABILITY DECISION

The server executes:

$$10^8$$

basic operations per second.

The nightly processing limit is:

$$10 \times 60 = 600 \text{ seconds}$$

Therefore, the maximum number of operations that can be performed within 10 minutes is:

$$10^8 \times 600$$
$$\boxed{60{,}000{,}000{,}000 \text{ operations}}$$

M1:

For:

$$n = 1{,}000{,}000$$
$$C(n) = \frac{1{,}000{,}000(999,999)}{2}$$
$$\boxed{499{,}999{,}500{,}000 \text{ operations}}$$

Running time:

$$\frac{499{,}999{,}500{,}000}{10^8} = 4{,}999.995 \text{ seconds}$$
$$= \boxed{83.33 \text{ minutes}}$$

Therefore:

$$\boxed{\text{M1 fails the 10-minute limit}}$$

M2:

For:

$$n = 1{,}000{,}000$$
$$k = \lfloor \log_2(999,999) \rfloor = 19$$

Therefore:

$$A(n) = 2^{20} - 1$$
$$\boxed{1{,}048{,}575 \text{ operations}}$$

Running time:

$$\frac{1{,}048{,}575}{10^8}$$
$$\boxed{0.0105 \text{ seconds}}$$

Therefore:

$$\boxed{\text{M2 passes the limit}}$$

S2:

For:

$$n = 1{,}000{,}000$$

approximately:

$$\boxed{20 \text{ comparisons}}$$

Running time:

$$\frac{20}{10^8}$$
$$\boxed{0.0000002 \text{ seconds}}$$

Therefore:

$$\boxed{\text{S2 easily passes the limit}}$$

M4 – Current Design:

The current design has:

$$T(n) = \Theta(n^2)$$

For:

$$n = 1{,}000{,}000$$
$$n^2 = 1{,}000{,}000{,}000{,}000$$

Therefore:

$$\boxed{1{,}000{,}000{,}000{,}000 \text{ operations}}$$

Running time:

$$\frac{10^{12}}{10^8} = 10{,}000 \text{ seconds}$$
$$= \boxed{166.67 \text{ minutes}}$$

Therefore:

$$\boxed{\text{Current M4 fails the limit}}$$

M4 – Proposed Design:

The proposed design has:

$$T(n) = \Theta(n\log_2 n)$$

For:

$$n = 1{,}000{,}000$$
$$\log_2(1,000,000) \approx 19.93$$

Therefore:

$$n\log_2 n \approx 1{,}000{,}000(19.93)$$
$$\boxed{\approx 19{,}931{,}569 \text{ operations}}$$

Running time:

$$\frac{19{,}931{,}569}{10^8}$$
$$\boxed{\approx 0.199 \text{ seconds}}$$

Therefore:

$$\boxed{\text{Proposed M4 passes the limit}}$$

M5 – Sales Analytics Engine:

The complexity is:

$$\Theta(n\log^2 n)$$

For:

$$n = 1{,}000{,}000$$
$$\log_2(1,000,000) \approx 19.93$$

Therefore:

$$n\log_2^2 n \approx 1{,}000{,}000(19.93)^2$$

$$\boxed{\approx 397{,}267{,}426 \text{ operations}}$$

Running time:

$$\frac{397{,}267{,}426}{10^8}$$
$$\boxed{\approx 3.97 \text{ seconds}}$$

Therefore, the analytics engine itself passes the 10-minute limit.

M5 – Nightly Report Generator:

The report generator performs:

$$f(n) = 3n^2 + 10n\log_2 n + 50$$

For:

$$n = 1{,}000{,}000$$
$$f(n) = 3(1{,}000{,}000)^2 + 10(1{,}000{,}000)\log_2(1{,}000{,}000) + 50$$

Using:

$$\log_2(1{,}000{,}000) \approx 19.9316$$

we get:

$$f(n) \approx 3{,}000{,}000{,}000{,}000 + 199{,}315{,}686 + 50$$
$$\boxed{f(n) \approx 3{,}000{,}199{,}315{,}736 \text{ operations}}$$

Running time:

$$\frac{3{,}000{,}199{,}315{,}736}{10^8}$$
$$\approx 30{,}001.99 \text{ seconds}$$
$$\approx 500.03 \text{ minutes}$$
$$\boxed{\approx 8.33 \text{ hours}}$$

Therefore:

$$\boxed{\text{M5 report generator fails the 10-minute limit}}$$

FINAL SCALABILITY COMPARISON

Module	**Complexity**	**Operations at n = 1,000,000**	**Approx. Running Time**	**Result**
M1	Θ(n²)	499,999,500,000	83.33 min	**FAIL**
M2	Θ(n)	1,048,575	0.0105 sec	**PASS**
S2	Θ(log n)	~20	0.0000002 sec	**PASS**
M4 Current	Θ(n²)	1,000,000,000,000	166.67 min	**FAIL**
M4 Proposed	Θ(n log n)	~19,931,569	0.199 sec	**PASS**
M5 Analytics	Θ(n log² n)	~397,267,426	~3.97 sec	**PASS**
M5 Report Generator	Θ(n²)	~3,000,199,315,736	~500.03 min	**FAIL**

RECOMMENDATIONS

M1 – Duplicate Listing Detector

M1 must be redesigned because its $\Theta(n^2)$complexity results in approximately 83.33 minutes for one million products. A hash-based approach using seller and product title as the key can reduce the average running time to approximately $\Theta(n)$.

M2 – API Rate Limiter

M2 does not need redesign. Its actual complexity is $\Theta(n)$, and it requires only about 0.01 seconds for one million requests. The team's $\Theta(n \log n)$claim should be corrected.

S2 – Product Search

S2 should be retained because binary search requires only $\Theta(\log n)$comparisons. It is highly suitable for a large sorted product catalogue.

M4 – Recommendation Engine

The current M4 design must be redesigned. Its $\Theta(n^2)$complexity requires approximately 166.67 minutes. The proposed $\Theta(n \log n)$design is significantly more scalable and should be implemented.

M5 – Sales Analytics Engine

The analytics engine has $\Theta(n\log^2 n)$complexity and remains manageable for the given workload. However, the nightly report generator must be redesigned because its $\Theta(n^2)$complexity results in approximately 8.33 hours of processing time.

CONCLUSION

The scalability audit shows that the main problem for ShopSphere is the use of quadratic-time algorithms for large datasets. M1, the current M4 design, and the M5 nightly report generator exceed the required 10-minute processing window when the catalogue reaches one million items.

M1 should be replaced with a hash-based duplicate detection method. M4 should use the proposed non-replicated $\Theta(n \log n)$approach. The M5 report generator should also be redesigned using a more efficient aggregation or indexing technique. M2 and S2 are sufficiently efficient and do not require redesign.

Therefore, the final recommendation is:

Redesign M1, M4 Current Design, and M5 Report Generator

while:

Retain M2, S2, and the M5 Analytics Algorithm

This analysis demonstrates the importance of asymptotic analysis in predicting whether an algorithm will remain practical when the input size increases from 100,000 to 1,000,000.

Rubrics – Application-Based Question

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand